# Sentinel AI

# 智慧產線全知守護與極速響應系統

## 成果說明書

### | 團隊名稱

**再想一下**

### | 競賽主題

☑ AI × 智慧現場　□ AI × 生成設計　□ AI × 營運大腦自主

### | 團隊成員


國立中山大學　資訊管理學系　碩士二年級　楊翊愷

國立中山大學　資訊管理學系　碩士二年級　陳啓祥

國立中山大學　資訊管理學系　碩士二年級　賴昱霖

國立高雄大學　資訊工程學系　學士四年級　林吟蓁


### | 指導老師


國立中山大學　資訊管理學系　康藝晃

# 成果摘要

## 01 問題

要解決什麼問題？

製造現場出現異常時，攝影機畫面、機台數據和人員班表分散在不同系統，警報響了以後，要先靠人確認是不是誤報、再打電話找負責的人，到場後還要靠老師傅的經驗判斷原因。排除故障時，若沒有確實停機、上鎖，很容易發生夾傷、捲入等職災。處理的經驗多半留在個人腦中，人員異動或新廠上線時無法延續，造成停機時間拉長、修復品質不一致。

---

## 02 解決方案

提出什麼方法？

Sentinel AI 是一套通用的智慧產線監控與維修協作平台：同時整合攝影機影像與機台數據來確認異常、判斷嚴重度，再依班表即時通知負責人員，並附上可能原因與對應 SOP。維修過程由 AI 助理依步驟引導，作業許可、上鎖掛牌等安全程序由系統強制把關，高風險步驟須主管確認；結案後的處理經驗經主管核准，回寫至知識庫供日後查詢。只要更換工廠設定檔與故障模式清單，就能套用到不同產線，本次以沖壓、銲接、組裝三條產線驗證。

---

## 03 AI 應用

AI 在方案中實際做什麼？

判斷分三層：規則引擎做確定的判斷（約 1 毫秒）；判斷模型 Jev 看送料口畫面判斷卡料、排出故障原因機率、檢查維修步驟的風險（約 0.5 秒）；LLM（DeepSeek）寫說明、陪技術員排查、整理結案案例。另以 YOLO 偵測未戴安全帽，再由 Jev 確認以減少誤報。

---

## 04 目前成果

Prototype、測試、數據或 Demo 進度。

完成可運行原型：後端 13 個模組、前端 7 個畫面（含技術員手機版），涵蓋 3 條產線、7 個場景，卡料場景以真實 AI 服務從偵測跑到知識回寫。實測：故障原因第一名命中 97.3%（37 個保留案例），安全帽判斷 120 張全對，警報到通知技術員約 0.5 秒；224 項自動化測試與 5 項端到端測試全數通過。

# 目錄

零、現場案例：沖床送料卡料 ..... 3

一、成果與關鍵數字 ..... 4

1. 完成項目 ..... 4

2. 相關數字 ..... 4

3. 初選改動 ..... 5

二、問題定義與應用情境 ..... 5

1. 應用場域 ..... 5

2. 現場痛點 ..... 5

三、系統架構與技術說明 ..... 6

1. 核心設計：三層判斷 ..... 6

2. 第一層：規則引擎 ..... 7

3. 第二層：Jev 判斷模型 ..... 8

4. 第三層：LLM 維修助理 ..... 9

5. 系統架構 ..... 10

6. 其他模組 ..... 11

7. 安全、法規與治理 ..... 12

8. 使用的 AI 工具與模型 ..... 13

四、系統運作流程展示 ..... 14

1. 場景二：沖床送料卡料 ..... 14

2. 場景一：人員未戴安全帽 ..... 18

3. 場景三：畫面上機台沒動，數據卻說在運轉 ..... 18

4. 其他場景（展示「換設定檔就能跑另一條產線」） ..... 18

5. 技術員手機畫面 ..... 19
五、系統測試報告 ..... 19
1. 測試方法 ..... 19
2. 自動化測試 ..... 20
3. 實驗結果 ..... 20
4. 限制 ..... 25
六、效益量化分析 ..... 25
1. 價值來源 ..... 25
2. 停機與產能試算（一條沖壓線，三種情境） ..... 26
3. 導入成本（一條線，假設值） ..... 27
4. 工安效益（可量測項目） ..... 27
七、落地可行性與永續價值 ..... 28
1. 從原型到工廠 ..... 28
2. 資訊安全 ..... 28
3. 永續與治理 ..... 29
八、與初選構想的差異 ..... 29
九、限制 ..... 30
附錄 A：名詞解釋 ..... 30
附錄 B：資料來源與授權 ..... 31
附錄 C：模擬資料聲明 ..... 33

# 零、現場案例：沖床送料卡料

夜班，沖壓線的送料機突然停了，一片鐵板卡在模具入口。送料口的攝影機拍到卡料，同一時間機台數據顯示送料速度歸零。系統把這兩個訊號對在一起，確認真的卡料，約 0.5 秒就送出派工通知。這時現場還沒有人發現送料機停了。

通知送到當班技術員小林的手機，系統查過當天班表，只通知他，不是全廠廣播。通知裡已經附上：

- 最可能的前三個原因和各自的機率（第一名是送料滾輪磨損）；
- 對應的維修 SOP。

小林走到沖床前，看著那片板子，在手機上問維修助理：「可以直接伸手拿出來嗎？」幾秒後助理回覆：

> **不行，不要直接伸手。**
>
> 1. 確認上鎖掛牌完成，機台已停止
> 2. 請主管到場確認
> 3. 用長柄夾具把料片取出，手不要進模具入口
> 4. 檢查送料滾輪有沒有磨損
> 5. 確定機台周圍沒人
> 6. 撤下上鎖牌
> 7. 重啟機台

上面是系統回覆格式的示意。第 5～7 步的復機是系統固定附在回覆最後的，由當初上鎖的人執行，不是 AI 叫人開機。

我們想縮短的是「警報響了」到「機器安全修好」中間這段時間：找人、猜原因、翻 SOP 由系統先做，危險動作能不能做，交給人確認。

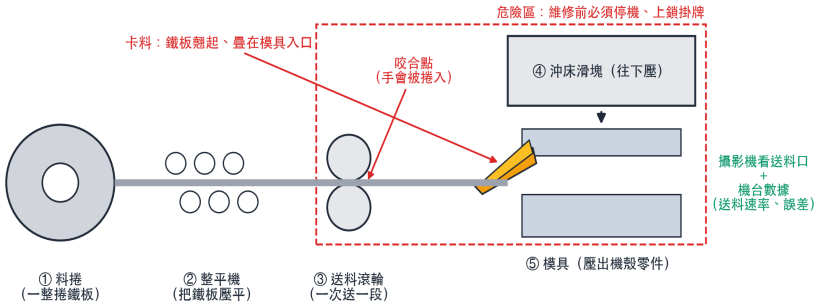



圖 0 沖床送料口示意圖：卡料位置、咬合點與危險區

# 一、成果與關鍵數字

## 1. 完成項目

| 項目 | 成果 | 狀態 |
|---|---|---|
| 網頁系統 | 後端 13 個模組、前端 7 個畫面（含技術員手機版） | 可運行 |
| 持續成長 | 偵測 → 告警 → 派工 → 現場維修 → 結案回報 → 主管核准後寫回知識庫（加進可查詢的案例，不是重新訓練模型） | 卡料場景已用真實 AI 服務從頭跑到尾 |
| 異常辨識 | 沖壓：卡料、影像與數據不一致、人員進入危險區銲接：機器人停止後仍冒煙組裝：電動起子扭力異常全廠：氣壓不足多台停機、未戴安全帽 | 7 個場景（3 條產線） |
| 安全設計 | 作業許可 → 上鎖掛牌 → 維修，順序不能跳；上鎖前要碰機台的步驟會標「需先完成上鎖掛牌」，危險步驟要主管確認；AI 不碰電源和安全裝置、不會叫人重新開機；發生職災時倒數 8 小時通報期限並準備草稿，由人確認後送出 | 程式強制 |
| 知識庫 | 3 條產線、40 種故障模式、40 份 SOP、6 份上鎖掛牌檢查表、240 筆模擬維修案例、法規原文 | RAG 知識庫（向量 + 關鍵字混合檢索）原始文字檔保留供人工審核 |
| 實驗 | 9 組實驗：找相似案例、判斷故障原因、機率準不準、看圖辨識、從異常到通知要多久、安全帽偵測模型，以及一組用新資料的再驗證 | 全部完成 |

表 1　完成項目

## 2. 相關數字

| 項目 | 結果 | 說明 |
|---|---|---|
| 找出故障原因 | 97.3% | 系統列出的「最可能原因」跟實際原因一樣的比例。只用固定規則判斷是 78.4%。這 37 個案例也拿來調整過系統，數字可能偏高，所以另外用 125 個沒拿來調整的模擬情況再測一次，全部判對 |
| 判斷有沒有戴安全帽 | 100% | 用公開資料集裡 120 張真實工作現場照片測試，系統判斷有沒有戴安全帽，一張都沒錯 |

| 項目 | 結果 | 說明 |
|---|---|---|
| 機台警報到通知技術員 | 約 0.5 秒 | 機台一發出卡料警報，不到一秒就通知到當班技術員的手機，通知裡附上最可能的三個原因和各自的機率 |

表 2　關鍵數字

### 3. 初選改動

- **判斷分成三層**：固定規則先處理；再由判斷模型 Jev（專門做判斷題的 AI，能看圖也能看數據，回答機率、不寫文章）快速補判；大型語言模型（LLM，像 ChatGPT 這類會寫文章的 AI）只負責說明和對話。
- **「最可能原因」附上機率，而且實測過準不準**，不再是 LLM 自己寫出來的百分比。
- **依台灣職安法規補上安全流程**：作業許可、上鎖掛牌、職災通報，AI 只能讀資料、不能控制機台。

## 二、問題定義與應用情境

### 1. 應用場域

以主辦贊助商勤誠興業所屬的伺服器機殼製造為情境：鈑金沖壓 → 折彎 → 銲接 → 表面處理 → 人工組裝 → 測試。

本原型實作沖壓、銲接、組裝三段的知識庫，展示以沖壓為主。

> 沖壓是什麼？ 把一捲鐵板（料捲）攤平後，由送料機一段一段送進沖床，沖床用模具「壓」出機殼零件。卡料就是鐵板沒有送正、翹起來或疊在一起，卡在模具入口。

系統是通用平台：只要工廠有「監視器影像」和「機台數據紀錄」兩種資料就能導入。換產線只要換一份工廠設定檔和故障模式清單，不用改程式。

### 2. 現場痛點

| 痛點 | 現場實況 | 影響 |
|---|---|---|
| 找人慢 | 機台警報響了，但該叫誰？夜班人少、一人顧多台，常常要打電話、廣播找人 | 停機時間被拉長 |

| 痛點 | 現場實況 | 影響 |
|---|---|---|
| 修得慢 | 排查靠老師傅經驗，新人常要試好幾輪才找到原因 | 平均修復時間（MTTR）偏高 |
| 資料互相矛盾 | 影像和機台數據分開看，例如「機台其實停了，系統還顯示運轉中」沒人發現 | 稼動率（機台實際在生產的時間比例）統計不準 |
| 經驗帶不走 | 老師傅的經驗在腦袋裡；勤誠正擴建馬來西亞、美國新廠 | 新廠上手慢 |
| 修的時候受傷 | 卡料時最危險的動作是「沒停機就伸手進去拿」 | 夾傷、壓傷事故 |

表 3　現場痛點

> 說明：沖壓機台本身多半會跳警報，操作員也常在附近。所以這套系統主要幫的不是「更早發現」，是發現之後的事：派給對的人、帶著原因方向、用安全的方式修好。

# 三、系統架構與技術說明

## 1. 核心設計：三層判斷

**用醫院急診來比喻：**

| 急診 | Sentinel AI | 速度 | 做什麼 |
|---|---|---|---|
| 檢傷分類（看到大出血直接推進去） | 規則引擎 | 毫秒 | 確定的、攸關安全的判斷，例如「送料速率歸零 + 畫面看到卡料 → 確認卡料」 |
| 護理師快速判斷「應該是腸胃炎，先掛內科」 | Jev 判斷模型 | 約 0.5 秒 | 從固定選項中挑出最可能的答案，**附上機率** |
| 醫師詳細問診 | LLM 維修助理 | 數秒 | 寫說明、跟技術員一問一答、整理成紀錄 |

表 4　三層判斷與急診分工對照

**為什麼要分三層？**

- 全部交給 LLM：速度不穩定（兩次實測裡，最慢的那 5% 回覆分別要等 46 秒和 5 秒），還可能亂說，安全告警不能等。
- 全部寫成規則：狀況太多，寫不完。
- 分三層：派工只等規則和 Jev（不到 1 秒），LLM 的說明晚幾秒再補上，不會拖慢派工。

LLM 不是萬能的，它的原理是依照前面的文字，一個字一個字接出「接下來最可能出現的字」（技術上稱為 decoder）。LLM 會寫通順的說明，但並不是在計算或查證，可能把不存在的規格講得很有把握。即使是能看圖的 LLM 也一樣。一般來說，專項能力（例如有沒有戴安全帽）比不上專項模型（例如 YOLO）。因此我們不讓 LLM 做判斷，判斷交給規則和專職模型，LLM 只負責把結果講清楚、陪技術員排查。

## 2. 第一層：規則引擎

規則引擎是我們用 Python 自行實作的條件判斷（if / else），不經過 AI 模型，因此反應快（約 1 毫秒）、結果可重現，每次命中哪一條規則都會寫進稽核紀錄。系統持續接收機台數據（送料速率、電流、警報碼）與影像判讀結果，規則引擎以滑動時間窗取最近數十秒的資料，逐條比對規則條件；門檻值集中在設定檔（config.yaml），可依現場調整，不寫死在程式裡。

目前 Demo 有 7 條規則：

| # | 規則 | 觸發條件 | 嚴重度 |
|---|---|---|---|
| 1 | 沖床送料卡料 | 送料速率在 30 秒內降為 0 AND Jev 判斷卡料機率 > 0.7 連續 2 張（Jev 無法使用時改為：送料速率降為 0 AND 影像判定機台靜止） | S2 |
| 2 | 畫面和數據對不上 | 影像判定機台靜止 AND 機台數據回報運轉中，持續 20 秒 | S2 |
| 3 | 人員進入危險區 | 影像偵測到人員在危險區 AND（機台運轉中 OR 同線沖床光幕停用 / 異常） | S3 |
| 4 | 未戴安全帽 | YOLO 偵測到頭部 AND 頭部與安全帽框重疊 < 0.3；同一人 3 秒內不重複通報 | S2 |
| 5 | 鎖附扭力異常 | 鎖附結果連續 NOK ≥ 3 秒 AND 近 100 次 NOK 率 > 3% | S2 |
| 6 | 銲接工位外冒煙 | （工位外有煙霧 AND 機器人已停止）OR 偵測到火光 | S3 |
| 7 | 多台同時停機 | 不同連動群組的 2 台以上機台在 15 秒內停止 AND 氣源壓力 < 4.8 bar | S3 |

表 5　規則引擎的 7 條規則

規則命中之後：

- **雙重確認才派工**：只有一種來源說有問題，不會派工，避免單一感測器誤判就亂發警報。卡料規則在 Jev 連不上時，會退回用「送料歸零 + 畫面不動」判斷，系統照樣運作。
- **同一件事不重複開單**：同一台機器、同一種問題還沒處理完，就更新原本那張事件單。

嚴重度三級：

- S1：記錄即可

- S2：需要人員處理，包含單一機台或連動機台停機
- S3：有人員受傷、起火等立即危險，或沒有連動關係的多台機台同時停止（例如全廠氣壓不足）

**分流**：工安類（未戴安全帽）只派工 + 附規範，不叫 LLM；設備類才啟動 AI 診斷。

**派工**：依當天班表決定。設備類派當班技術員並通知現場主管，工安類派現場主管，S3 通知該線職位最高者與職安人員。門禁顯示人不在廠就改派，沒人接單就逐級往上（技術員 → 現場主管 → 組長 → 課長）。

## 3. 第二層：Jev 判斷模型

> **Jev 是什麼？** 新創公司 TypeSafe 在 2026 年 9 月 15 日才推出的「判斷模型」（decision model），是很新的一類 AI。跟 ChatGPT 這類會寫文章的 AI 不同，Jev 只回答選擇題、是非題、評分題，約 0.5 秒回傳答案和每個選項的機率，而且不會回答選項以外的東西。我們用的是 2026 年 10 月 14 日發布的 1.14 版，可以讀圖片，這一版還新增了在圖上框出目標位置的功能。

以卡料事件舉例：

| 系統問 Jev | 題型 | Jev 回答 |
|---|---|---|
| 附上送料口畫面：「板材有沒有堆疊、歪斜，或卡在送料口沒有正常送入？」 | 是非題 + 框位置 | 是，機率 0.82，並框出卡住的板材 |
| 附上機台數據：「最可能的原因是哪一個？」選項是這條產線的故障模式清單 | 選擇題 | 第一名「送料滾輪磨損 / 打滑」，機率 0.93；前三名和機率一起附在派工通知裡 |
| 安全把關：維修助理（LLM）給的每一步，再交給 Jev 檢查。例如「用夾具從側邊把料片取出」：「這一步會不會進入機台運轉範圍？」 | 是非題 | 會，機率 0.71 → 標紅、等主管確認（0.55 ~ 0.7 之間只標黃提醒注意） |

表 6　Jev 判斷舉例（卡料事件）

> **機率「準不準」是什麼意思？** 就像氣象預報：如果氣象局說「降雨機率 70%」的日子，真的大約七成有下雨，這個預報就是「準的」。我們用一批刻意不放進知識庫、專門拿來考試的案例（下文稱「保留案例」）檢查了 Jev 的機率準不準（第五章第 3 節）。

在系統裡，派工前一次請求同時問 4 題：事件類別、是否誤報、嚴重度、最可能的原因（選項來自該產線的故障模式清單）。

## 4. 第三層：LLM 維修助理

LLM 放在最後一層，只做它擅長的事：把規則和 Jev 的判斷結果寫成技術員看得懂的說明，並在現場一問一答，帶著技術員一步步排查。它不負責判斷「有沒有出事」或「原因是什麼」。

一張工單一個助理，只負責這台機器、這個故障。可以查 SOP、查機台數據、找相似案例、記錄每一步結果。

以下限制寫在程式裡強制執行，不是靠提示詞要求 LLM 遵守：

1. **工單流程不能跳**：主管核准作業許可（依廠內規定，可設定是否需要）→ 逐項完成上鎖掛牌 → 進入維修。故障原因、SOP、相似案例、機台數據，以及讀警報碼、看操作面板、看攝影機畫面這類不用碰機台的檢查，技術員在任何階段都能問助理。
2. **要碰機台的步驟，沒上鎖就標出來**：助理給的每一步都會判斷「要不要先上鎖掛牌」。還沒完成上鎖掛牌時，伸手、開護罩、拆零件這類步驟會標成「需先完成上鎖掛牌」，並提醒技術員先去上鎖。讀警報碼、看數據趨勢、看攝影機畫面這類只看不碰的步驟，由程式直接判定不需要把關，不會被誤標。我們不把資訊整段鎖住，是因為那樣反而會逼人為了看步驟先把清單亂勾完，紀錄就不可信了。
3. **每一步都經過安全把關**：LLM 只是在接字，可能寫出危險的步驟自己卻不知道，所以系統不直接相信它的輸出，而是把每個建議步驟再交給 Jev 回答四個是非題：要不要先上鎖掛牌、會不會進入機台運轉範圍、會不會碰到高壓電、是不是要通電才能做。結果分兩級：

**注意（黃）**：可能靠近危險區，但技術員保持距離就能避開，畫面提醒，不必叫主管。

**需主管確認（紅）**：碰到高壓電、需要通電，或進入危險區的機率很高（0.7 以上），主管用自己的身分確認後才算數。

4. **不叫人重新開機**：助理不會說「可以復電」。每則維修回覆最後，程式會固定附上復機步驟：由當初上鎖的人自己確認周圍沒人、工具都拿出來，拆下自己的鎖，再重新啟動機台。
5. **不能改工單狀態**：工單狀態只能由人在畫面上操作。




圖 1 上鎖掛牌完成前問維修助理：只看不碰的檢查照常給，要碰機台的步驟標成「需先完成上鎖掛牌」

## 5. 系統架構

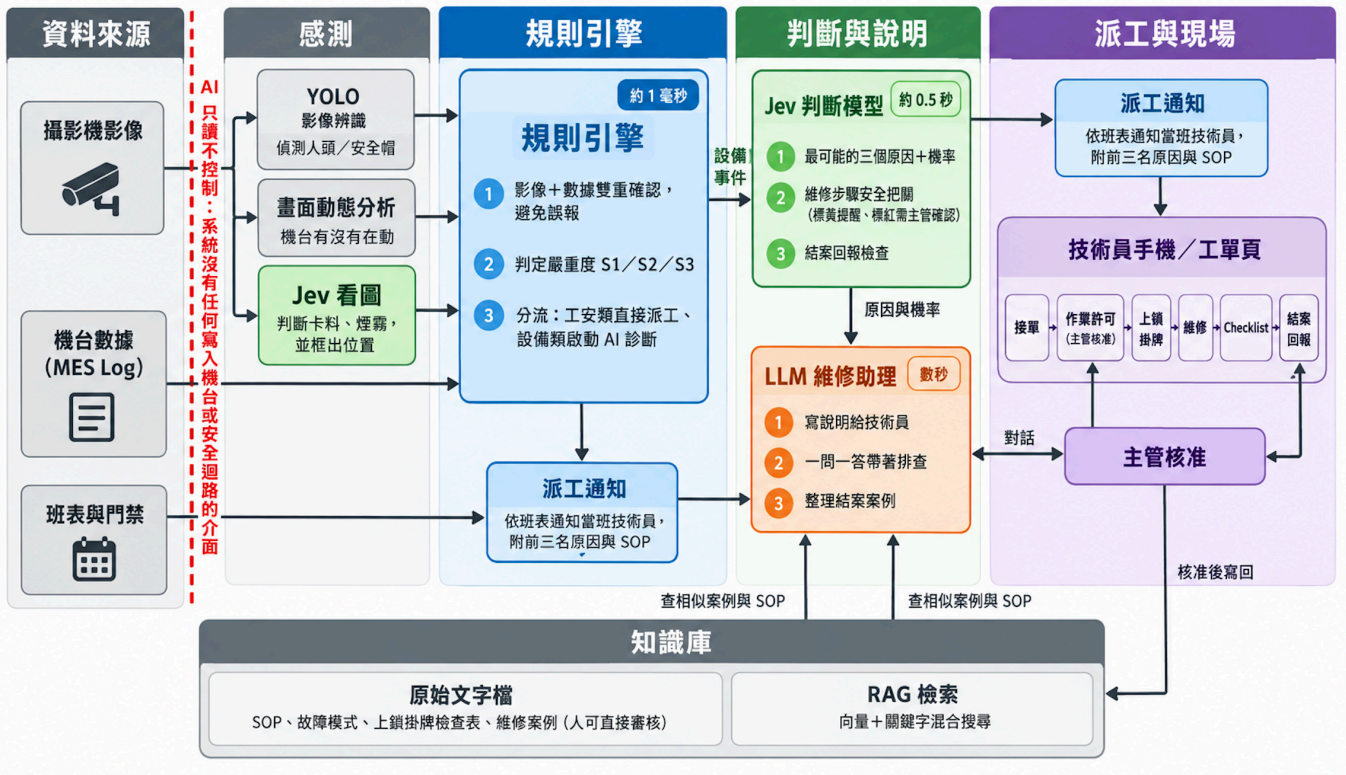



圖 2 系統架構：規則（藍）、Jev（綠）、LLM（橘）、人（紫）；AI 只讀不控制

| 層 | 內容 |
|---|---|
| 資料來源 | 攝影機、機台數據紀錄（MES Log）、班表與門禁（原型中皆為模擬） |
| 感測 | YOLO（影像辨識模型）找出人頭和安全帽；分析畫面中機台有沒有在動；Jev 看關鍵畫面；機台數據超過門檻就標記 |
| 規則 | 決定嚴重度；確認影像和數據說法一致；分成工安類或設備類；決定派給誰；沒人接單就往上通知 |
| Jev 快速判斷 | 最可能原因、嚴重度補判、看圖確認、維修步驟安全把關、結案回報檢查 |
| LLM 深入說明 | 給技術員看的說明、維修助理對話 |
| 派工與閉環 | 派工通知、任務看板、Checklist、結案回報、知識回寫、稽核紀錄 |

表 7　系統架構各層內容

# 6. 其他模組

### 6.1 影像 + 機台數據

- **安全帽偵測（兩段式）**：先用 YOLO 物件偵測模型在畫面中找出人頭和安全帽；如果偵測到「沒戴」，再把頭部裁切圖（臉部先模糊）送給 Jev 確認。為什麼要兩段？YOLO 很快，但該抓到的安全帽只抓到約六成（召回率 0.59，第五章第 3 節 E7），只靠影像辨識會把有戴的人誤報成沒戴；Jev 確認的正確率實測 100%（E4），兩段合起來可以壓低誤報。
- **卡料偵測**：卡料沒有公開的訓練照片，沒辦法像安全帽那樣訓練偵測模型。所以我們不訓練模型，而是每 2 秒把送料口的畫面送給 Jev 問「板材有沒有卡住？」，Jev 回答機率，並在畫面上框出卡料的大概位置，方便人一眼看到（框的位置準不準沒有另外測試，派工也不看框，只看機率和機台數據）。機台數據出現異常時會立即多送一張。之後的做法見第九章。
- **機台數據判定**：送料速率、送料長度誤差、馬達電流、溫度、警報碼。所有數字比較由程式計算，結論再交給 Jev（Jev 不擅長算數）。

### 6.2 查資料：知識庫

故障模式清單、SOP、上鎖掛牌檢查表、模擬維修案例、法規與公開指引原文。

分兩層：

- 原始文字檔：每份 SOP、案例、檢查表都是文字檔，人可以直接打開閱讀、修改、審核。

- RAG 檢索：從文字檔切段建立索引，讓 AI 回答前先查到相關段落再回答，而不是憑記憶回答。索引隨時可以從文字檔重建。

  兩種查法：

- 找相似案例：依意思找（向量搜尋）加上依字面找（關鍵字搜尋）一起用，同機型、同事件、訊號相同的案例加分（實測前 5 筆相似案例裡有正確原因的比例，從 84% 提高到 100%）。
- 找 SOP：直接用 Jev 選出的故障模式編號查，像圖書館用書號找書，同一個原因一定拿到同一份 SOP，不會因為字面相似而拿錯。

### 6.3 派工與結案

派工通知：嚴重度、機台、前三名原因與機率、SOP 編號、工單連結；LLM 說明幾秒後補上。

結案：Checklist 勾完 → 技術員寫回報 → Jev 檢查有沒有寫清楚「原因、處置、結果」→ LLM 整理成一筆案例草稿 → 主管核准後才寫進知識庫。核准時只把這一筆加進去，下次同類故障就找得到這筆經驗。

## 7. 安全、法規與治理

| 規範 | 系統怎麼做 |
|---|---|
| 職業安全衛生設施規則第 57 條：現行條文（113 年 8 月 1 日版）要求維修前停止運轉及送料、上鎖或設置標示；116 年 1 月 1 日施行的修正條文另要求危害防止計畫與作業許可程序 | 符合現行條文，並提前對齊修正條文：先核准作業許可 → 逐項完成上鎖掛牌 → 才進入維修；上鎖完成前，需要碰機台的步驟會被標成「需先完成上鎖掛牌」 |
| 由上鎖人親自解除上鎖（參考美國 OSHA 29 CFR 1910.147(e)(3)）；電路作業送電前應確認無感電之虞（設施規則第 254 條第 2 項） | AI 不宣告可以復電；解除程序由人執行 |
| 機械設備器具安全標準、ISO 13849：沖床安全靠法定安全裝置與連鎖 | **AI 只讀、不控制**：系統沒有任何寫入機台或安全迴路的介面 |
| 職業安全衛生法第 37 條：發生死亡、3 人以上罹災、1 人以上需住院等職災，雇主應於 8 小時內通報；非經許可不得移動或破壞現場 | 符合條件時啟動 8 小時倒數，預填通報草稿、提醒保留現場、通知職安與急救人員；由組長以上的人確認後送出（判斷傷亡由人標記，AI 不從畫面推斷）。原型的「送出」只在系統內記錄送出的人和時間，還沒有串接勞動檢查機構的通報管道 |
| 個人資料保護法第 8 條（告知義務）、第 19 條（蒐 | 人臉模糊；只送機台區域圖到雲端；門禁與派工紀錄只 |

| 規範 | 系統怎麼做 |
| --- | --- |
| 集要件）；勞動部對職場監控的溝通原則 | 用於流程追溯，不做個人考核；導入前應告知員工並與勞工代表溝通 |

表 8　法規要求與系統對應做法

> 法定義務的主體是雇主，AI 不是守法主體。Sentinel AI 的角色是協助雇主落實這些規定：看見風險、叫對的人、留下紀錄，最後由人做決定。AI 是第二道眼睛，不是第一道防線。

## 8. 使用的 AI 工具與模型

| 環節 | 工具 / 模型 | 為什麼選它 |
| --- | --- | --- |
| 安全帽偵測 | YOLO（Ultralytics yolov8s），以 SH17 + Hard Hat Workers 微調為「頭部 / 安全帽」兩類 | 輕量、筆電可即時推論（每張約 2.4 毫秒） |
| 快速判斷、看圖、安全把關 | Jev 1.14（TypeSafe，經 OpenRouter） | 約 0.5 秒、附機率、不會答非所問；每 1 萬次約 US$1.4（約新台幣 43 元，一次問 4 題）。目前只能用雲端 API，沒有公開權重 |
| 說明與對話 | DeepSeek V4.1 Flash（經 Ollama Cloud；開源權重，可架設在廠內地端主機） | 繁中品質好；維修助理約 1 秒開始出字、2 秒回完；Ollama 雲端與本機同一套介面，可移到廠內主機（完整模型權重約 511 GB，地端要用多張高階 GPU 的伺服器） |
| 知識庫 | Chroma 向量資料庫 + 多語文字向量模型（bge-m3）+ 關鍵字搜尋 | 免架設、中英文資料混用、筆電可跑（每次查詢約 0.1 秒）；資源不夠時可換成 e5-small，準確度幾乎一樣 |
| 系統 | Python FastAPI（後端）/ Next.js（前端） | 即時推播、手機也能用 |

表 9　使用的 AI 工具與模型

# 四、系統運作流程展示

## 1. 場景二：沖床送料卡料

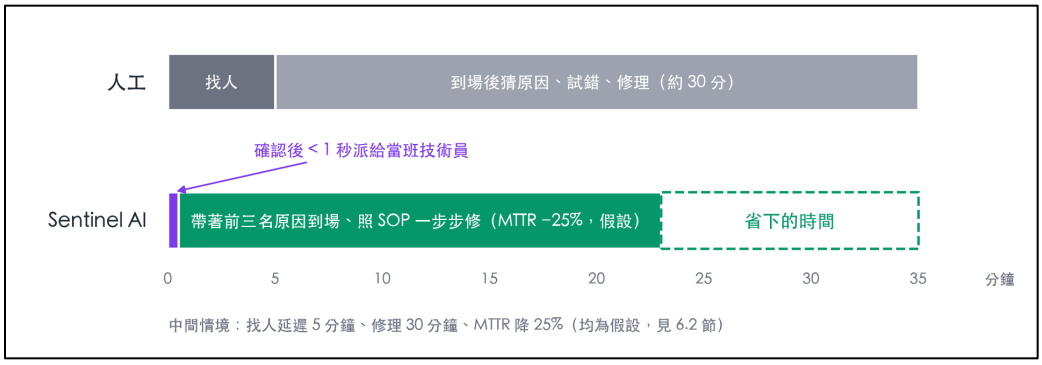



圖 3 人工流程與 Sentinel AI 的時間比較（中間情境，假設值）

| 步驟 | 發生什麼事 | 畫面 |
|---|---|---|
| ① | 送料長度誤差逐步變大、送料速率歸零、跳出送料異常警報 | 圖 4 |
| ② | Jev 看送料口畫面判定卡料，並框出板材位置 | 圖 4 |
| ③ | 規則雙重確認 → S2 設備類 → Jev 判斷前三名原因 → 派給當班技術員 | 圖 5 |
| ④ | LLM（DeepSeek）的說明補上（大意）：誤差是慢慢變大，不是突然跳掉，最可能是送料滾輪磨損 | 圖 6 |
| ⑤ | 技術員接單 → 申請作業許可 → 主管核准 → 逐項勾完上鎖掛牌 | 圖 7 |
| ⑥ | 維修助理一步一步引導；技術員問「可以直接伸手拿出卡住的料片嗎？」→ 助理回答不行，改用長柄夾具並請主管到場 | 圖 8 |
| ⑦ | Checklist 勾完 → 寫回報 → Jev 檢查完整 → 主管核准 → 寫入知識庫 | 圖 9 |

表 10　卡料場景執行流程

**實測時間**（實際呼叫 Jev 和 DeepSeek，重複 5 次，E6）：

| 區段 | 由誰處理 | 一般情況（中位數） | 較慢時（最慢 5%） |
|---|---|---|---|
| 送料警報 → 規則確認卡料 | 系統程式（規則引擎） | 0.02 秒 | 0.03 秒 |
| 規則確認 → 派工通知（含前三名原因） | Jev 判斷原因與機率；系統程式查班表、送出通知 | 0.51 秒 | 0.62 秒 |
| 派工通知 → 說明補上 | LLM（DeepSeek）寫給技術員看的說明 | 5.4 秒 | 8.5 秒 |

表 11　卡料場景實測時間（E6）

> 註：從畫面「看得到卡料」到確認，目前約 **4** 秒，主要在等機台數據顯示送料速率歸零。這是刻意的「雙重確認」，用幾秒換取不誤報。

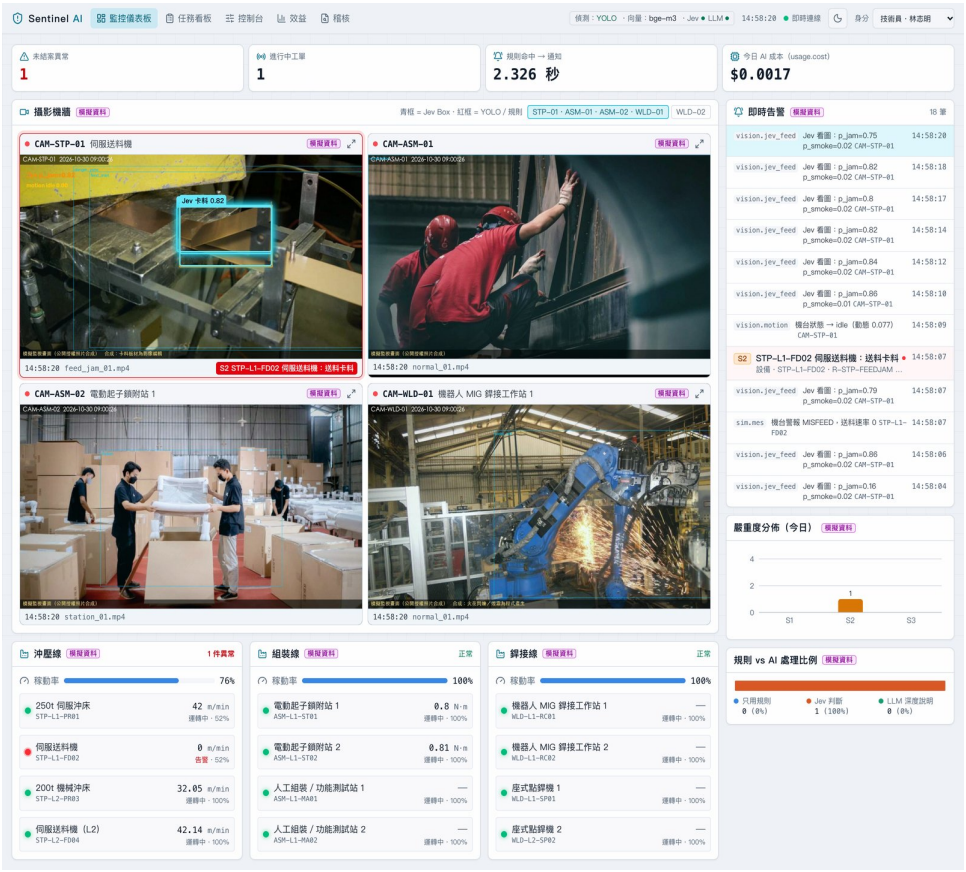



圖 4 監控儀表板：送料口畫面由 Jev 框出卡料位置，即時告警列出 S2 事件

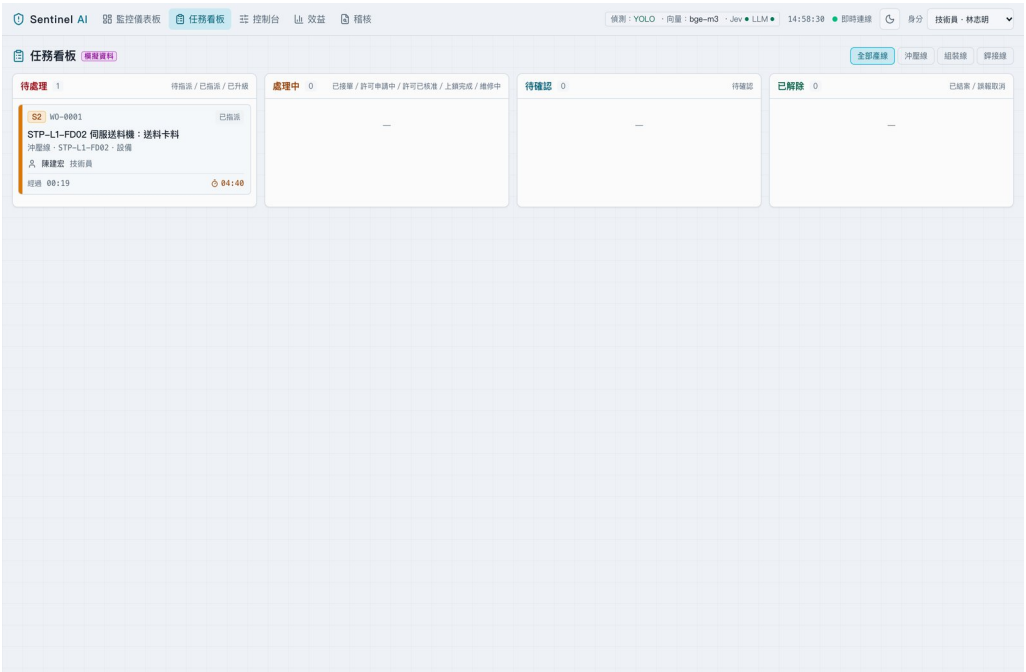



圖 5 任務看板：工單已派給當班技術員，顯示嚴重度與接單倒數

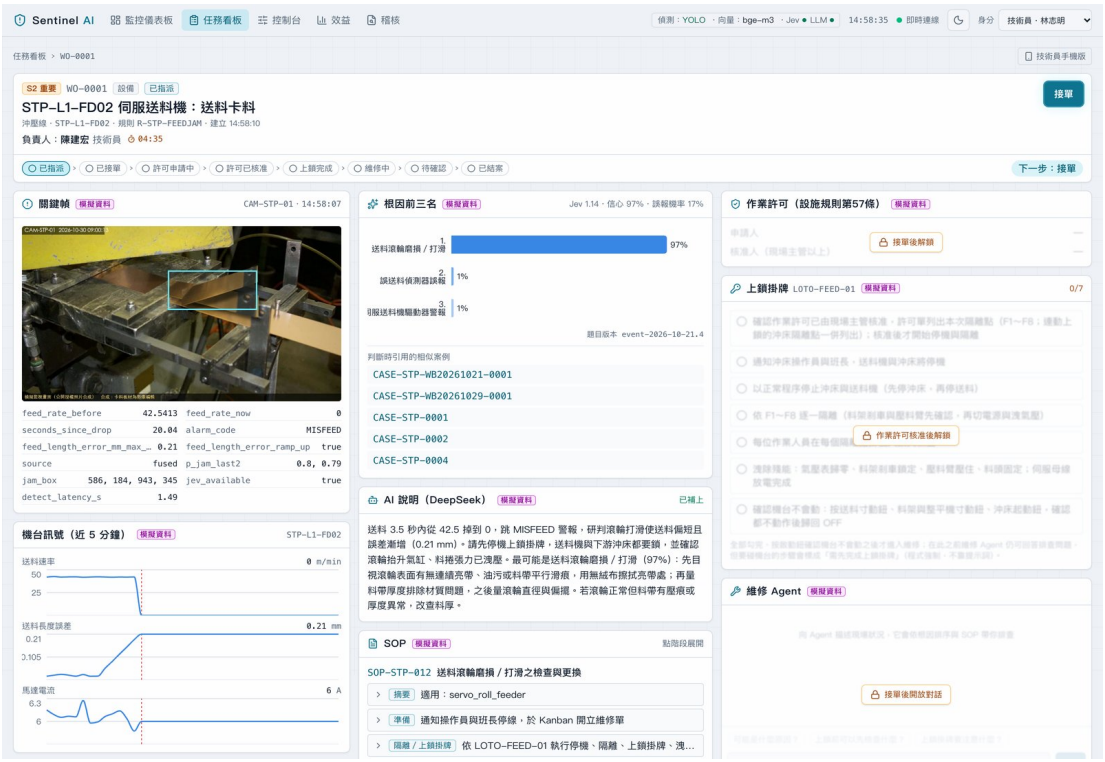



圖 6 工單詳情：關鍵畫面、機台訊號、前三名原因與機率、DeepSeek 說明、引用的相似案例




圖 7 作業許可由主管核准後，才解鎖上鎖掛牌清單




圖 8 維修助理（上鎖完成後）：風險高的步驟標紅、要主管確認，風險低的標黃提醒；最下方是程式固定附上的完工復機步驟

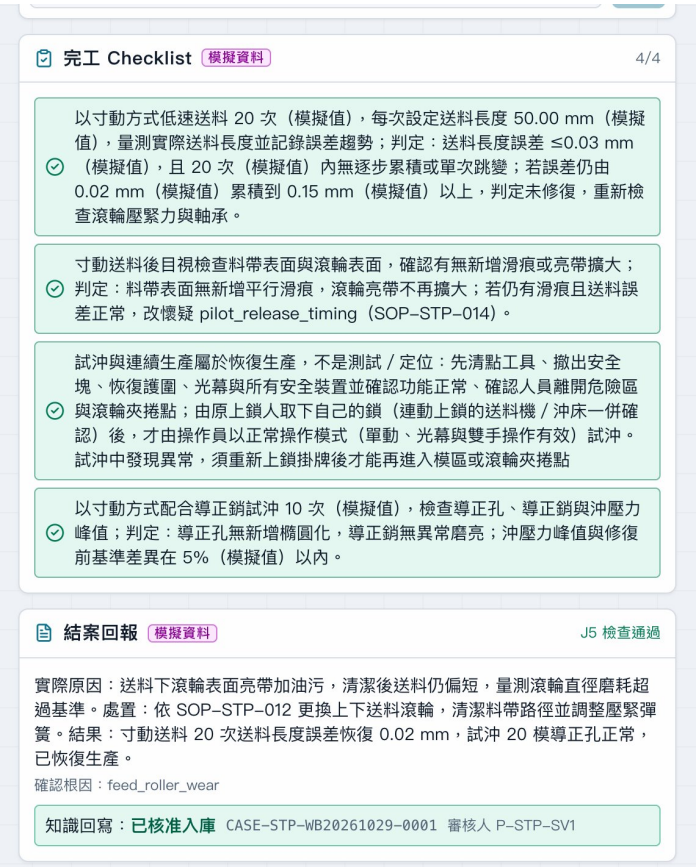



圖 9 完工 Checklist、結案回報（Jev 檢查通過）與知識回寫「已核准入庫」

## 2. 場景一：人員未戴安全帽

YOLO 偵測到人頭沒有安全帽 → Jev 看頭部裁切圖確認 → S2 工安類 → 不叫 LLM，直接派給該線現場主管並附 PPE 規範。這個場景刻意不叫 LLM：沒戴安全帽沒有原因需要推理，派工就好。

> 我們選安全帽，是因為它有公開授權的訓練資料。實際上伺服器機殼廠多數區域要求的是安全鞋、護目鏡、防割手套、耳塞；系統架構相同，只要換成對應的偵測類別即可。

## 3. 場景三：畫面上機台沒動，數據卻說在運轉

影像判斷機台靜止，但機台數據回報運轉中 → 判定數據或感測器失準（S2），避免錯誤資料影響稼動率與排程。

## 4. 其他場景（展示「換設定檔就能跑另一條產線」）

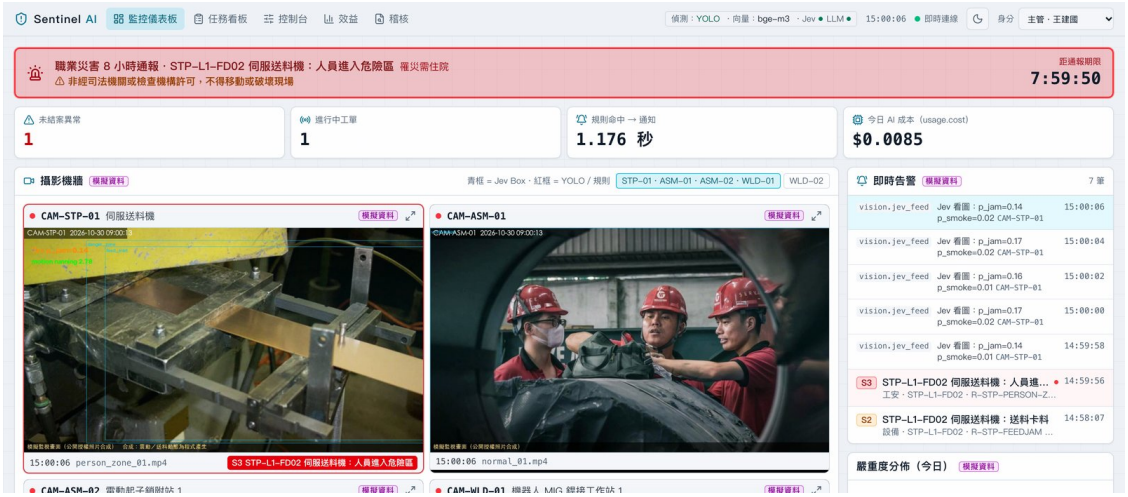



圖 10 S3 人員進入危險區：主管標記傷亡後，儀表板出現職災 8 小時通報倒數與「不得移動現場」提醒

| 場景 | 產線 | 發生什麼事 | 系統反應 |
|---|---|---|---|
| 人員進入危險區 | 沖壓 | 光幕被設為靜音時有人進入模具區 | S3 緊急：通知該線職位最高者、職安與急救人員；若主管標記有人受傷住院，啟動 8 小時通報倒數 |
| 機器人停止後仍冒煙 | 銲接 | 銲接工位以外、或機器人停止後偵測到煙霧（銲接時的火花是正常的） | S3 緊急通報 |
| 電動起子扭力異常 | 組裝 | 起子回報鎖附不合格（NOK）次數增加 | S2 + AI 判斷原因（離合器、批頭磨損、螺絲滑牙等） |
| 全廠氣壓不 | 跨產線 | 不同產線、沒有連動關係的多台機台 | S3（同一組連動機台一起停，例如送料機卡料 |

| 場景 | 產線 | 發生什麼事 | 系統反應 |
|---|---|---|---|
| 足 | | 同時停止 | 時沖床跟著停，仍算 S2） |

表 12　其他場景

## 5. 技術員手機畫面




圖 11 技術員手機畫面

# 五、系統測試報告

## 1. 測試方法

| 編號 | 測什麼 | 資料 | 狀態 |
|---|---|---|---|
| T0 | 自動化測試：規則、派工、工單狀態、安全限制 | 程式內建 | 完成 |
| R1 | 相似案例找得準不準 | 37 筆保留測試案例 | 完成 |
| E1 | Jev 判斷「最可能原因」準不準 | 37 筆保留測試案例（從未進入知識庫） | 完成 |
| U1 | 用沒參與調整的資料再驗證一次 | 125 筆新產生的合成情境 | 完成 |
| E2 | 純規則 vs Jev vs LLM 比較 | 同 E1 | 完成 |

| 編號 | 測什麼 | 資料 | 狀態 |
|---|---|---|---|
| E3 | Jev 的機率準不準（校準） | E1 + 70 筆補充情境 | 完成 |
| E4 | Jev 看照片判斷有沒有戴安全帽 | 公開資料集 SH17 的真實照片 120 張 | 完成 |
| E5 | Jev 看畫面判斷卡料 | 公開授權照片 + 合成圖 | 完成（真實卡料未驗證） |
| E6 | 從異常到通知要多久 | 場景重播（真實 AI 服務） | 完成 |
| E7 | 安全帽偵測模型（YOLO） | SH17 + Hard Hat Workers 驗證集 | 完成 |

表 13　測試項目一覽

# 2. 自動化測試

224 項後端自動化測試、5 項「在畫面上實際點按鈕」的端到端測試全數通過，包含 3 個場景完整重播、Jev 故障時照常派工、誤報處理、逐級升級、門禁不在廠時改派、工單狀態流程、作業許可與上鎖掛牌先行、復電語句過濾、安全把關、結案檢查、稽核紀錄不可修改。另有 4 項測試實際呼叫 Jev 與 DeepSeek，也全數通過。端到端測試會開瀏覽器，用切換身分、點按鈕、在對話框打字的方式，把卡料、職災通報、未戴安全帽、銲接冒煙、手機版五個流程完全測試。

# 3. 實驗結果

### 3.1 R1：相似案例找得準不準

Jev 判斷原因時，會參考知識庫裡最像的 5 筆歷史案例，所以案例找得準不準，直接影響判斷。

| 檢索方式 | 第 1 筆就是正確原因 | 前 5 筆有正確原因 |
|---|---|---|
| 第一版（只依意思找，而且只找同機型） | 73% | 84% |
| 改良後（意思 + 關鍵字一起找，同機型只加分、不限定） | 100% | 100% |

表 14　R1 相似案例檢索結果

第一版漏掉的主要原因是「硬性限定同機型」：例如沖床「氣壓不足」的案例，知識庫裡只出現在送料機的紀錄中，限定同機型就找不到。

### 3.2 E1 / E2：最可能原因判斷

| 方法 | 第一名就猜對的比例 | 回應時間（中位數） | 每 1 萬次成本 |
|---|---|---|---|
| 純規則（取該類事件最常見的原因） | 78.4%（29/37） | < 1 毫秒 | 0 |
| Jev 1.14（第一版檢索） | 86.5%（32/37） | 0.46 秒 | US$1.4 |
| Jev 1.14（改良後檢索） | 97.3%（36/37） | 0.46 秒 | US$1.4 |
| DeepSeek V4.1 Flash（改良後檢索） | 100%（37/37） | 1.26 秒（最慢 5%：5.1 秒） | 訂閱制 |
| Jev，沒參與調整的 125 筆合成情境（U1） | 100%（125/125） | — | — |

表 15　E1 / E2 最可能原因判斷結果

進步幾乎都來自「找對相似案例」。相似案例找對，Jev 和 LLM 都能正確回答。

37 筆保留案例是我們拿來調整檢索參數的那一批，所以 97.3% 偏樂觀。我們另外產生 125 筆從沒拿來調整的情境，結果一樣很好。不過這批是程式直接產生，沒有人寫的雜亂文字，比保留案例簡單。

兩批都是模擬資料，不能代表真實工廠的準確度。

Jev 和 LLM 準確度接近，Jev 的優勢在穩：同一案例重跑 3 次答案完全一致、選項順序打亂也不影響；LLM 的回應時間會忽快忽慢（另一次實測最慢的 5% 要 46 秒）。所以派工只等 Jev，LLM 的說明放到派工之後。

**Jev 缺點**：

- **事件類別**：Jev 67.5%，純規則 90%，所以系統以規則為準。
- **是否誤報**：2 筆誤報案例 Jev 都沒抓到，誤報主要靠規則的雙重確認來擋。
- **清單以外的新原因**：1 筆不在故障模式清單裡的案例，Jev 仍硬選了一個既有原因，所以「以上皆非」這個選項還要再調整。

### 3.3 E3：Jev 的機率準不準

檢查方式：把 Jev 對每個案例給的「第一名原因機率」，跟實際答案比對。如果 Jev 說 0.9 的那些判斷，真的大約九成是對的，代表它給的機率可以信。

| Jev 給的機率 | 案例數 | 實際答對 | 怎麼解讀 |
|---|---|---|---|
| 0.8 以上 | 35 | 97% | Jev 很有把握時幾乎都對，可以直接附在派工通知 |
| 0.8 以下 | 2 | 2 個都對 | 案例太少，看不出準不準 |

表 16　E3 Jev 機率校準結果

改良檢索後，Jev 給出 0.8 以上機率的判斷，實際有 97% 是對的（35 個案例）。整體來看，Jev 說的機率和實際答對率平均只差約 2 個百分點（校準誤差 ECE 0.02）；沒拿來調整的 125 筆是 0.03。

第一版檢索時，Jev 給 0.6～0.8 的判斷實際只對 20%，也就是太有自信，所以門檻定在 **0.8**：低於 0.8 時派工照常送出，但同時請 LLM 深入分析。

### 3.4 E4：Jev 看照片判斷安全帽

| 指標 | 結果 |
|---|---|
| 正確率 | **100%**（120/120；有戴、沒戴各 60 張） |
| 回應時間 | 一般 0.47 秒，較慢時（最慢 5%）0.59 秒 |
| 框出「沒戴安全帽的頭」 | 70% 的框和正確位置的重疊度（IoU，1 代表完全重疊）在 0.5 以上；90% 在 0.3 以上 |
| 每 1 萬次成本 | 約 US$0.6（約新台幣 19 元） |

表 17　E4 Jev 判斷安全帽結果

### 3.5 E7：安全帽偵測模型（YOLO）

召回率是「該抓到的安全帽，有幾成真的抓到」；mAP50 是物件偵測常用的綜合分數，0～1，越高越好。

| 驗證集 | 類別 | 原始模型（SH17） | 微調後 |
|---|---|---|---|
| SH17（1,620 張） | 安全帽 召回率 | 0.53 | 0.59 |
| | 安全帽 mAP50 | 0.67 | 0.69 |
| | 頭部 mAP50 | 0.88 | 0.90 |
| | 頭部 召回率 | — | 0.86 |
| Hard Hat Workers（945 張） | 安全帽 召回率 | — | 0.92 |
| | 安全帽 mAP50 | — | 0.97 |

表 18　E7 安全帽偵測模型（YOLO）結果

SH17 驗證集只有 93 張含安全帽，目標小、角度多變，安全帽召回率只有 0.59；在 Hard Hat Workers 這類作業現場照片上是 0.92。漏抓安全帽會讓有戴的人被判成沒戴（多報），這部分由 Jev 看頭部裁切圖確認擋下（E4，120 張全對）。真正會漏報的是頭部沒被偵測到，頭部召回率為 0.86，也就是約一成多的人頭沒被抓到。導入後會用廠內攝影機畫面再微調，並改用較大的模型和較高的訓練解析度。

### 3.6 E5：Jev 看畫面判斷卡料

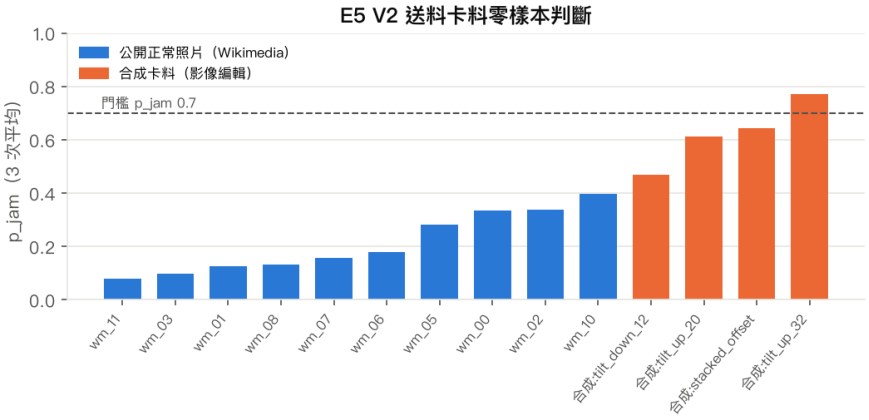



圖 12 正常照片與合成卡料圖的卡料機率

| 測試 | 結果 |
|---|---|
| 10 張公開授權的正常送料機照片 | **0 張誤判為卡料**，0 張誤判為煙霧 |
| 4 張合成卡料圖（影像編輯） | 只有 1 張超過門檻 0.7（4 張的卡料機率在 0.47 ~ 0.78；沒合成前的正常底圖是 0.13） |
| 回應時間 | 中位數 0.67 秒 |

表 19　E5 Jev 判斷卡料結果

Jev 不會把正常畫面亂判成卡料，但真實卡料的偵測率還沒有資料：公開的卡料照片找不到，合成圖又不夠逼真。這也是系統要「雙重確認」（畫面 + 機台數據）的原因：只靠畫面不會派工。

不過從圖上看得出一件事：4 張合成卡料圖雖然大多沒過門檻，但卡料機率（0.47 ~ 0.78）都比 10 張正常照片（0.08 ~ 0.40）高，代表 Jev 看得出卡料和正常的差別。之後拿到真的卡料畫面，再重新調整門檻。

### 3.7 E6：從異常到通知

見第四章第 1 節表格。另外未戴安全帽場景：畫面偵測到 → 派工通知 0.21 秒（中位數；加上 Jev 確認約再多 0.5 秒）。

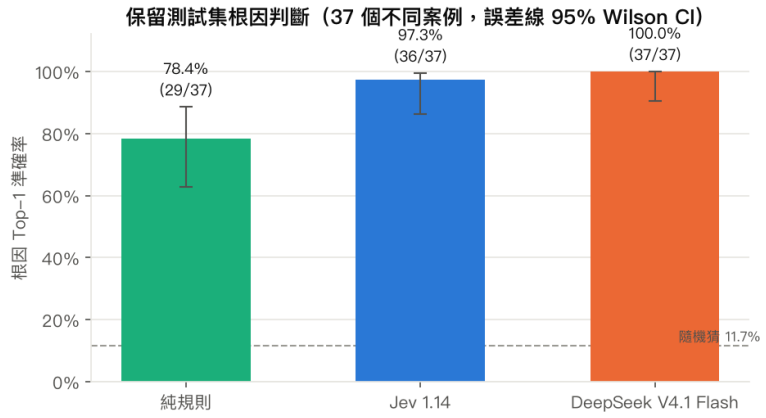



圖 13 根因第一名命中率：純規則、Jev、DeepSeek（95% 信賴區間）

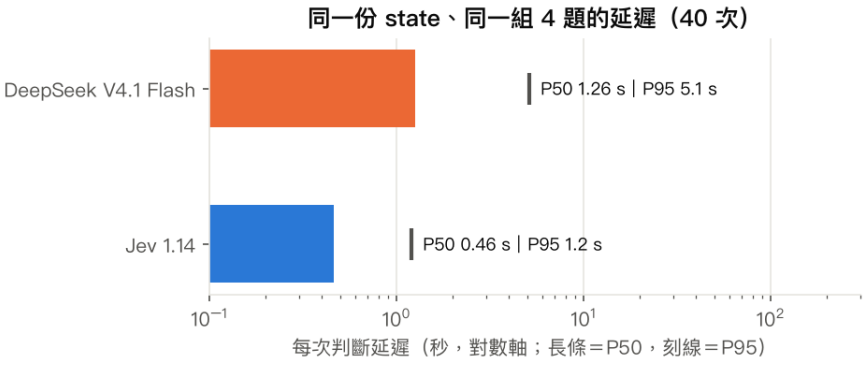



圖 14 E2 回應時間分布：Jev 穩定在 1 秒內，LLM 有長尾

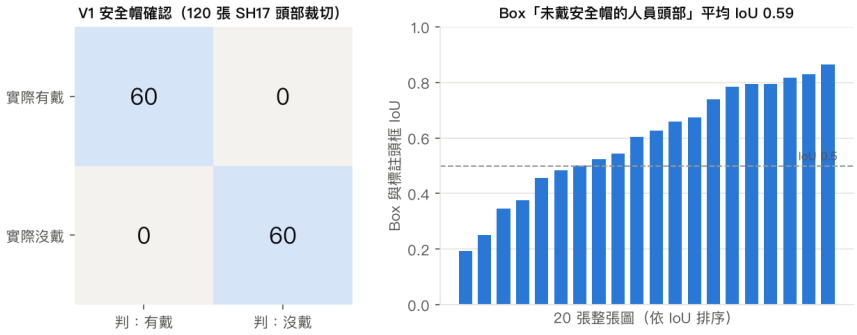



圖 15 E4 Jev 判斷安全帽（SH17 真實照片 120 張）

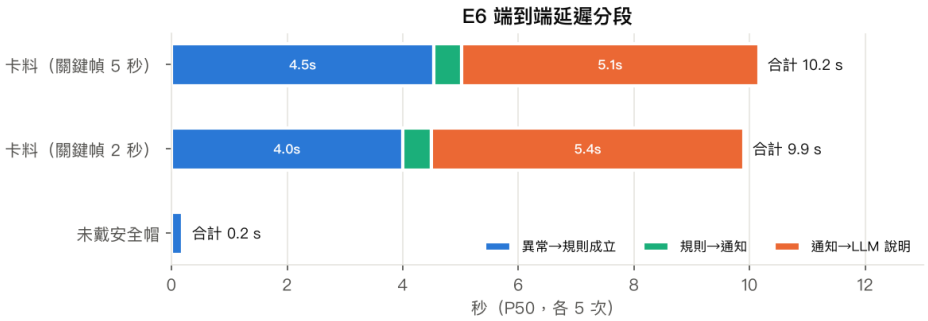



圖 16 E6 從異常到通知的各段時間

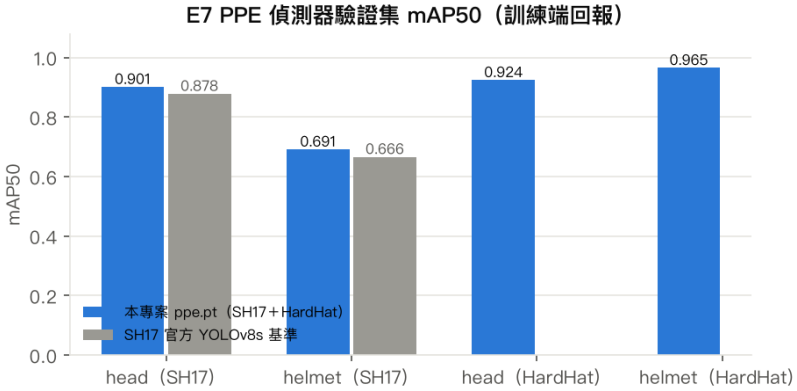



圖 17 E7 安全帽偵測模型微調前後

## 4. 限制

測試案例與知識庫案例由同一套程式產生（同分布），只能證明「流程與判斷介面可用」，不能代表真實工廠的準確度。

卡料沒有公開的真實照片，E5 只能用替代素材；正式導入前需要現場影片再驗證。

展示用的攝影機畫面是用公開授權照片做成的，卡料和煙霧是影像編輯合成的；YOLO 與 Jev 是真的在這些畫面上做偵測。

知識庫由 AI 扮演資深維修課長與設備保全工程師審核，不是真人專家審核；門檻數值標示為「模擬值」。

# 六、效益量化分析

## 1. 價值來源

| 環節 | 人工 | Sentinel AI | 依據 |
|---|---|---|---|
| 確認異常 | 機台警報 + 人工判斷是否誤 | 影像 + 數據雙重確認，警報後 0.02 秒成立 | E5 / |

| 環節 | 人工 | Sentinel AI | 依據 |
|---|---|---|---|
| | 報 | | E6 |
| 找到對的人 | 打電話、廣播、找主管 | 依班表直接派給當班技術員，約 0.5 秒；逾時自動升級 | E6 |
| 知道先查哪裡 | 到場後才開始猜 | 通知當下附前三名原因，第一名在模擬案例上 97% 命中 | E1 |
| 安全排除 | 靠個人習慣 | 作業許可 + 上鎖掛牌程式強制 | T0 |
| 經驗留存 | 在老師傅腦袋裡 | 每次結案經主管核准後寫回知識庫 | — |

表 20　人工與 Sentinel AI 的價值比較

## 2. 停機與產能試算（一條沖壓線，三種情境）

**共同假設**（均為假設值，導入時以廠方數據替換）：

計畫生產時間 16 小時 × 22 天 = 352 小時 / 月

每月需人員處理的停機 60 次；目前平均修復時間 30 分鐘

導入後「找到對的人」縮短為約 0.5 分鐘

每小時產值 NT$15,000

| 情境 | 目前每次「找人」延遲 | 修復時間（MTTR）縮短 | 每月省下的停機時數 | 每月效益 |
|---|---|---|---|---|
| 保守 | 2 分鐘 | 10% | 4.5 小時 | 約 NT$6.8 萬 |
| 中間 | 5 分鐘 | 25% | 12 小時 | 約 NT$18 萬 |
| 樂觀 | 12 分鐘 | 40% | 23.5 小時 | 約 NT$35 萬 |

表 21　停機與產能試算（三種情境）

修復時間縮短比例的參考：業界整理指出，結構化知識庫與引導式排查可縮短 MTTR 約 30–50%；設備廠商案例報告 40–65%（附錄 B，均非同儕審查研究）。我們刻意取 10% / 25% / 40% 三種情境。

設備綜合效率（OEE）影響：中間情境約提高 2.3 個百分點（以 OEE 60% 為基準；60% 為離散製造常見參考值，世界級約 85%，源自 Nakajima 的 TPM 架構）。

# 3. 導入成本（一條線，假設值）

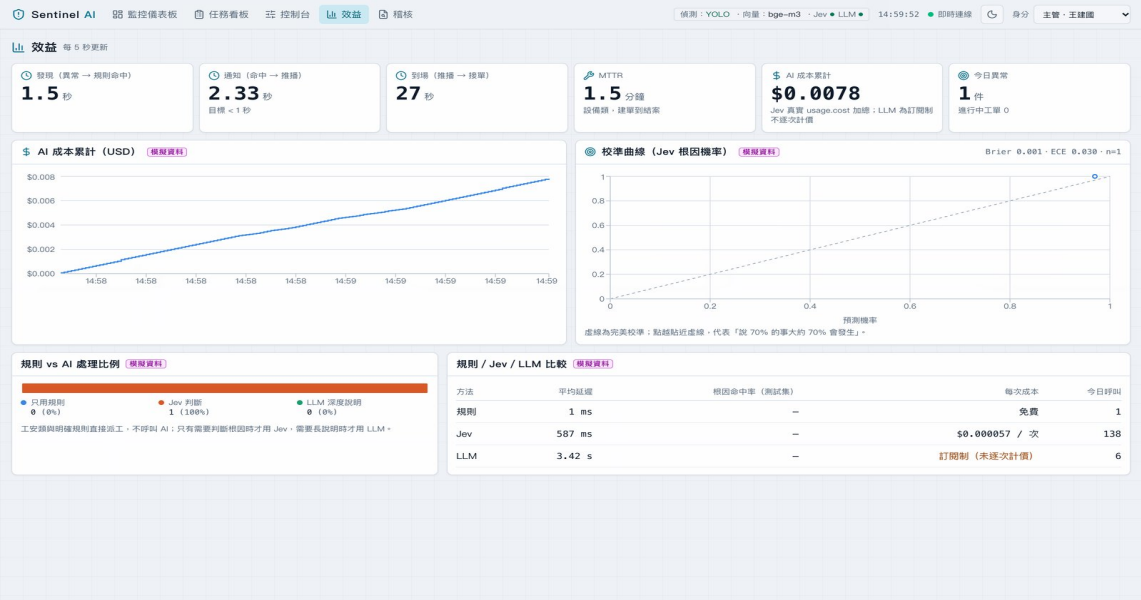



圖 18　系統效益頁：反應時間、MTTR、AI 實際成本累計

**系統建置**：後端是一般的 Python 服務，可以直接裝在工廠現有的伺服器上，不需要另外採購主機；攝影機沿用現有監視系統的影像串流，通知接廠內原有系統或通訊軟體（見第七章）。一次性的工作主要是人力：介接 MES 機台數據，以及把廠方的 SOP、歷史工單整理進知識庫，天數依各廠資料狀況而定。

**每月 AI 服務費用**：

| 項目 | 可選的服務 | 費用 |
|---|---|---|
| LLM（說明與對話） | Ollama Cloud（原型使用，月訂閱）；也可以換成 Claude、GPT 等，只需要改 LLM 介接的那一層，這類服務多半依用量計費 | 原型使用的方案約 NT$1,000 / 月 |
| Jev（判斷） | TypeSafe，依呼叫次數計費 | 每 1 萬次約 US$1.4；以每天 2,000 次估算約 NT$260 / 月 |

表 22　每月 AI 服務費用

和上一節每月 NT$6.8 萬到 35 萬的停機減少效益相比，AI 服務費用很小；導入時真正要評估的是介接與資料整理的人力。

# 4. 工安效益（可量測項目）

導入後可以持續統計，作為工安管理指標：

維修工單的作業許可核准率、上鎖掛牌完成率（系統強制，應為 100%）

維修助理攔截的危險步驟次數（例如「伸手進模具區」被標紅並要求主管確認，或標黃提醒注意）

職災通報是否在 8 小時內送出

# 七、落地可行性與永續價值

## 1. 從原型到工廠

| 元件 | 原型 | 正式導入 |
|---|---|---|
| 影像 | 影片模擬攝影機 | 換成攝影機串流網址，程式不用改 |
| 機台數據 | 模擬器 | MES 介接 |
| 事件佇列（系統內部傳遞事件的管道） | 程式內建 | 換成工廠常用的 Kafka，事件名稱已照它的格式命名 |
| LLM | Ollama Cloud | 廠內 Ollama 主機（同一套介面），資料不出廠 |
| Jev | 雲端，只送不含個人資料的事件代碼與機台區域圖 | Jev 沒有公開權重，只能用雲端；若工廠要求資料完全不出廠，可改用 Ollama 支援、介面與 Jev 相容的開源判斷模型，但準確度要重新測試 |
| 通知 | Discord（示範） | 優先接廠內原有系統的通知介面（例如 MES、產線 Andon 燈號）；沒有的話，用現場已在用的通訊平台（LINE 官方帳號、Teams 等） |
| SOP 與上鎖掛牌程序 | 模擬內容 | 匯入廠方已核定的版本；系統只負責取對的那份、逐項勾選、留紀錄 |
| 新增產線 | — | 一份工廠設定檔 + 一份故障模式清單 |

表 23　從原型到正式導入

## 2. 資訊安全

整段影片留在廠內，只送機台區域的關鍵畫面；人員畫面先模糊臉部。

送給 Jev 的是「沖壓線、送料速率歸零、保養 34 天前」這類事件代碼，不含人名、客戶、料號。

正式導入前需簽資料處理協議並評估跨境傳輸。

## 3. 永續與治理

| SDG | 本系統的貢獻 |
|---|---|
| SDG 8 尊嚴勞動 | 維修助理第一件事永遠是作業許可與上鎖掛牌，降低卡料排除時的夾傷事故；職災通報有倒數、有草稿 |
| SDG 9 產業創新 | 老師傅經驗變成可查詢、可複製的知識庫，支援勤誠馬來西亞、美國新廠上線 |
| SDG 12 負責任生產 | 送料偏差在變成大量不良品前就被處理，減少廢料 |
| SDG 16 可信賴治理 | 每個判斷都有稽核紀錄；安全決定由規則與人負責；不做個人考核 |

表 24　對應聯合國永續發展目標（SDG）

# 八、與初選構想的差異

| 初選寫法 | 第二階段做法 | 理由 |
|---|---|---|
| LLM 排出最可能原因與百分比 | Jev 判斷模型回傳機率，並實測準不準、機率可不可信 | LLM 寫的百分比沒有經過驗證 |
| 規則 / AI 比例 90/10 與 95/5 混用 | 改用實測：派工只等規則與 Jev，LLM 的說明在派工之後再補上 | 用量測取代宣稱 |
| 銲接段「偵測火花」為異常 | 「銲接工位以外的煙霧或火光」 | 銲接本來就有火花 |
| 沒戴安全帽也啟動 AI 診斷 | 工安類只派工、不叫 LLM | 沒有原因需要推理 |
| 依門禁指派負責人 | 依當天班表與職位派工，門禁只確認在不在廠 | 門禁不知道誰負責哪台 |
| 卡料需要大量訓練資料 | Jev 看關鍵畫面直接判斷 | 卡料沒有公開資料集 |
| （未提及） | 作業許可 → 上鎖掛牌 → 維修、職災 8 小時通報、AI 只讀不控制 | 對齊職安法規 |

表 25　初選構想與第二階段的差異

# 九、限制

| 項目 | 現況 |
|---|---|
| 影片素材 | 以公開授權照片製作，卡料與煙霧為影像編輯 |
| 卡料看圖準確度 | 缺真實照片 |
| 行動通知 | 網頁與終端機；Discord 介接程式已完成 |
| 銲接、組裝場景 | 已完成（銲接冒煙、起子扭力異常） |
| 專家審核 | AI 角色扮演審核 |
| 測試案例數量 | 只有 37 筆，數字的誤差範圍大 |

表 26　目前限制

**卡料偵測的長期做法（尚未實作）**

目前卡料完全靠 Jev 看圖，因為沒有卡料照片可以訓練模型。實際導入一間工廠後，可以分兩步換成廠內自己的模型：

**先用正常畫面訓練異常偵測模型**：正常運轉的畫面很好收集，每台機器錄幾天就夠。這類模型（例如 PatchCore）只用正常畫面訓練，現場畫面和正常差太多就報異常。它說不出是什麼異常，所以仍要搭配送料歸零等機台數據一起確認。

**累積真實卡料照片後訓練 YOLO**：YOLO 需要有框好的卡料照片才能訓練，光有正常畫面不行。每次卡料都會留下關鍵畫面，並經技術員結案確認，累積到足夠數量後，就能訓練專屬這台機器的 YOLO，在廠內主機即時偵測，不用連網也不用付 API 費用，Jev 改做複查。

# 附錄 A：名詞解釋

| 名詞 | 說明 |
|---|---|
| MES | 製造執行系統：記錄每台機器何時開、何時停、做了多少的系統 |
| Log / 機台數據 | 機台自動記錄的運轉紀錄，例如每分鐘沖幾下、溫度幾度 |
| OEE（設備綜合效率） | 機台真正有效生產的時間比例 |
| MTTR（平均修復時 | 從故障到修好平均要多久 |

| 名詞 | 說明 |
| --- | --- |
| 間） | |
| YOLO | 一種很快的影像辨識模型，能在畫面中框出人、頭、安全帽 |
| LLM（大型語言模型） | 會讀會寫的 AI，例如 ChatGPT、DeepSeek |
| Jev | TypeSafe 在 2026 年 9 月推出的判斷模型（decision model）：只做判斷、不寫文章，能看圖也能看數據，回傳答案和機率。本專案用 1.14 版 |
| 維修助理（Agent） | 會查資料、會一問一答、會記錄的 AI 助理 |
| 知識庫（RAG） | 讓 AI 回答前先查公司自己的資料（SOP、維修案例），而不是憑記憶回答；原始文字檔人可以直接審核 |
| 保留案例 | 刻意不放進知識庫、專門拿來測試的案例，就像考試題目不能先給學生看 |
| 上鎖掛牌（LOTO） | 維修前把機器的電源、氣壓等能量關掉並上鎖、掛上標示，防止有人誤開機 |
| 作業許可 | 危險作業前由主管核准 |
| 校準 | AI 說 70% 的事情，實際上大約七成會發生 |
| ECE（校準誤差） | AI 說的機率和實際答對率平均差多少，越小越好 |
| 關鍵畫面（關鍵幀） | 從影片中挑出來分析的單張畫面 |
| 召回率 | 該抓到的東西，有多少比例真的抓到 |
| IoU | 兩個框重疊的程度，1 代表完全重疊 |
| mAP50 | 物件偵測常用的綜合分數，0～1，越高越好 |
| P50 / P95 | 中位數 / 最慢的 5% 從這裡開始 |
| S1 / S2 / S3 | 嚴重度：記錄即可 / 需要人處理 / 有立即危險 |

表 27　名詞解釋

# 附錄 B：資料來源與授權

| 資料 | 用途 | 授權 |
| --- | --- | --- |
| SH17（製造業 PPE 資料集，影像來自 Pexels） | YOLO 預訓練、E4 測試 | CC BY-NC-SA 4.0 |
| Hard Hat Workers（Voxel51 版） | YOLO 微調 | CC0 1.0 |
| 全國法規資料庫 | 法規原文 | 政府公開資訊 |

| 資料 | 用途 | 授權 |
|---|---|---|
| eCFR / OSHA | 美國職安法規 | 美國政府著作（公有領域） |
| HSE INDG316 | 機械安全指引 | 英國開放政府授權 |
| NASA-STD-8739.4A | 線材組裝標準 | 美國政府著作 |
| OEE 參考值（Nakajima、TPM） | 效益試算背景 | 業界整理，見參考文獻 |
| MTTR 縮短參考值 | 效益情境 | 業界整理與廠商案例，非同儕審查 |

表 28　資料來源與授權

## 參考文獻


勞動部。（無日期）。*機械設備器具安全標準*。全國法規資料庫。https://law.moj.gov.tw

勞動部。（無日期）。*職業安全衛生法*。全國法規資料庫。https://law.moj.gov.tw

勞動部。（2024）。*職業安全衛生設施規則*（113 年 8 月 1 日修正）。全國法規資料庫。https://law.moj.gov.tw

Ahmad, H. M., & Rahimi, A. (2024). *SH17: A dataset for human safety and personal protective equipment detection in manufacturing industry* (arXiv:2407.04590). arXiv. https://arxiv.org/abs/2407.04590

Ahmad, H. M. (n.d.). *SH17 dataset* [Data set]. GitHub. https://github.com/ahmadmughees/SH17dataset

Elevat. (2026). *OEM case study: Reducing MTTR* [Case study]. https://www.elevat-iot.com/wp-content/uploads/2026/02/Elevat-Case-Study-OEM-Reducing-MTTR-1.pdf

Fabrico. (n.d.). *What is a good OEE score in manufacturing?* https://www.fabrico.io/blog/what-is-good-oee-score-manufacturing/

Health and Safety Executive. (n.d.). *Using work equipment safely* (INDG316). https://www.hse.gov.uk/pubns/indg316.pdf

Jocher, G., Qiu, J., & Chaurasia, A. (2023). *Ultralytics YOLO* (Version 8.0.0) [Computer software]. https://docs.ultralytics.com

Nakajima, S. (1988). *Introduction to TPM: Total productive maintenance*. Productivity Press.

Occupational Safety and Health Administration. (n.d.). *The control of hazardous energy (lockout/tagout)* (29 CFR 1910.147). Electronic Code of Federal Regulations. https://www.ecfr.gov/current/title-29/subtitle-B/chapter-XVII/part-1910/subpart-J/section-1910.147

Ollama. (n.d.). *Ollama Cloud* [Online service; DeepSeek V4.1 Flash]. https://ollama.com

OpenRouter. (n.d.-a). *Jev 1.14* [Model page]. https://openrouter.ai/typesafe/jev-1.14

OpenRouter. (n.d.-b). *Jev* [Documentation]. https://openrouter.ai/docs/guides/community/jev

Reliamag. (n.d.-a). *OEE benchmarks by industry: What world class really means*. https://reliamag.com/guides/oee-benchmarks-by-industry-what-world-class-really-means/

Reliamag. (n.d.-b). *Predictive maintenance ROI benchmarks: What the studies show*. https://reliamag.com/guides/predictive-maintenance-roi-benchmarks-what-the-studies-show/

Sentinel AI 團隊. (2026). *AI-Manufactory* [Source code and evaluation results]. GitHub. https://github.com/morris3927/AI-Manufactory

TypeSafe. (2026, September 15). *Introducing System One models and Jev* [Blog post]. https://typesafe.ai/blog/introducing-system-one-models-and-jev

Voxel51. (n.d.). *Hard hat detection* [Data set]. Hugging Face. https://huggingface.co/datasets/Voxel51/hard-hat-detection

Zerynth. (n.d.). *Technical knowledge management in industrial maintenance: Reducing MTTR*. https://zerynth.com/blog/technical-knowledge-management-industrial-maintenance-mttr/

# 附錄 C：模擬資料聲明

本原型使用的工廠設定、機台、人員姓名、班表、門禁、MES Log、維修案例、SOP 數值皆為團隊自行產生之模擬資料，數值標示「模擬值」；影像素材來自公開授權資料集或程式合成畫面。知識庫內容由大型語言模型依公開資料起草，經 AI 角色扮演審核，未經真人專家審核。